When Does Predicting Spot Interruptions Beat Checkpointing? A Trace-Driven Study with Real Reclaims

Angshuman Chakravertty
SVKM's NMIMS (Deemed to be University)
angshumanchakravertty2@gmail.com

MD Rayyan
SVKM's NMIMS (Deemed to be University)
rayyan1652@gmail.com

Abstract

Spot instances cost a fraction of on-demand capacity but can be reclaimed on two minutes' notice. We ask when predicting reclaims lets a training job do better than reacting to that notice or checkpointing on a schedule. We built Argus, a Kubernetes operator that checkpoints and migrates a job when a risk model raises an alarm, and tested it three ways. A trace-driven simulator replays real Spot price history with the model's own causal scores making the decisions. A real EKS cluster runs the whole path, and we measured in-region checkpoint upload at about 114 MB/s per node. A small fleet of Spot instances produced real reclaim labels. Reacting to the notice is nearly optimal while a checkpoint fits inside two minutes and fails once it does not. For 15 nodes with 300 second checkpoints, model alarms cut the extra time to finish from 34% (best periodic policy) to 14%. The catch is the label. The model predicts price spikes with AUROC 0.980, but on real reclaims it scores AUROC 0.54, while AWS's own placement score reaches 0.79. Prediction pays when its signal is real, and price history alone is not that signal.

1 Introduction

EC2 Spot capacity costs far less than on-demand but AWS can take it back on two minutes' notice [Amazon Web Services, 2026e]. A training job that ignores this loses everything since its last checkpoint. The loss grows with scale. If each of N nodes is reclaimed with probability p in an interval, the chance that at least one is reclaimed is $1-(1-p)^N$ (at $p = 0.05$ and $N = 16$ this is about 56%), and a synchronous job stops when any node does. Prior systems make training itself survive preemption [Athlur et al., 2022, Thorpe et al., 2023, Jang et al., 2023]. We ask a narrower policy question. Given a free notice and the option to checkpoint on a schedule, when is it worth predicting reclaims?

One answer is that it is never needed, because a checkpoint can simply be made fast enough to fit inside the notice. Section 3.1 shows where that stops being true. We measured checkpoint upload from EKS nodes to S3 at about 114 MB/s per node, so a model with 12 bytes of training state per parameter needs more than two minutes beyond roughly one billion parameters.

We make four contributions. First, a measured account of when the two-minute notice stops being enough (Section 3.1). Second, a trace-driven benchmark in which the predictor must actually predict, with the real model's causal scores driving every decision, non-learned baselines, an alarm budget chosen on a separate slice, and intervals resampled by whole days (Sections 3.2 and 3.3). Third, real reclaim labels from CloudTrail and a test of the price-based model against them (Section 3.4). Fourth, a real EKS run of the full path, including five defects it exposed in our own design (Section 3.5).

Preprint.

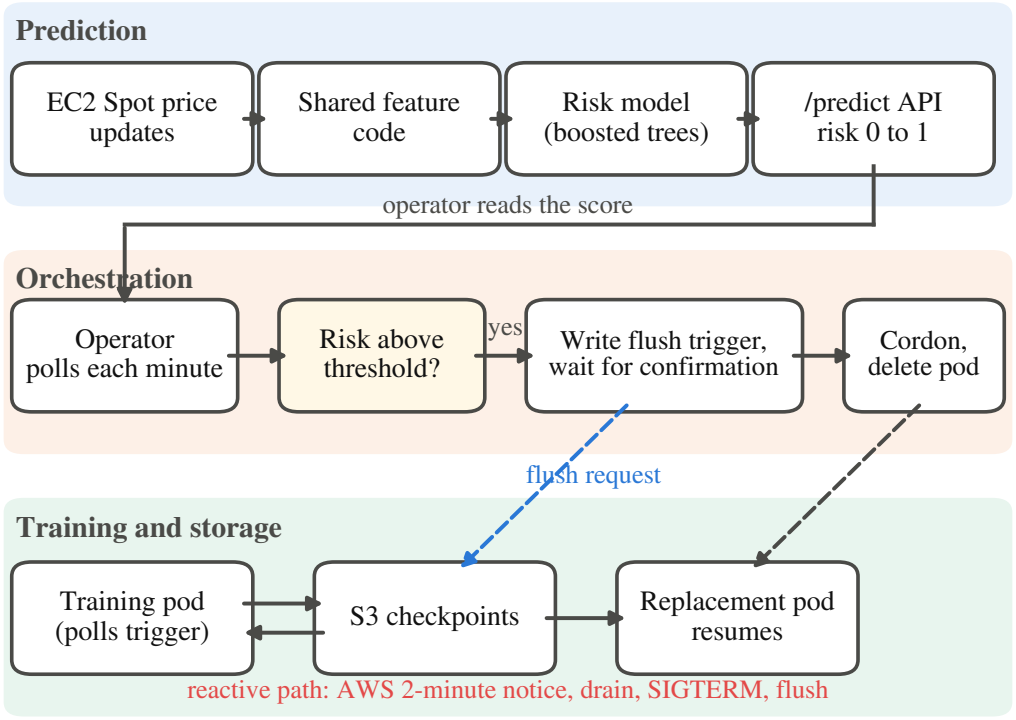


Figure 1: Argus architecture. The operator reads a risk score, asks the pod to flush, waits for an acknowledgement, and only then moves the job. The reactive path is the AWS two-minute notice.

2 System Design

Argus has three layers (Figure 1). The prediction layer turns recent Spot price updates into a calibrated risk score and serves it at `/predict`. Training and serving share one feature module, so the model sees the same inputs in both. The orchestration layer is a `kopf` operator [Dobies and Wood, 2020] for a `SpotResilientJob` resource. It polls the score every minute and, when the score crosses the threshold, asks the training pod to flush a checkpoint, waits for confirmation, cordons the node and deletes the pod so it is rescheduled. The training layer is a pod that checkpoints to S3 and resumes from the latest checkpoint. A second, reactive path uses the AWS Node Termination Handler [Amazon Web Services, 2026b], which drains the node when the real notice arrives and sends SIGTERM, and the pod's handler flushes a checkpoint.

The pod role in our cluster can read and write S3 objects but cannot delete them, which shaped the handshake. The operator writes a unique request id to a trigger object. The pod saves a checkpoint, and only if the upload succeeded writes the same id to an acknowledgement object. The operator waits for its id. After a migration the operator ignores alarms for 15 minutes, the prediction horizon, because the pool's risk usually stays high after the job has left it.

3 Evaluation

3.1 When the two-minute notice is not enough

A checkpoint is useful only if it completes before the node disappears. We uploaded files of 16 MB to 2 GB from the EKS worker nodes to S3 in the same region. Rates above 500 MB were between 114 and 150 MB/s per node regardless of the number of concurrent requests, and a 2 GB file took 19 seconds. Table 1 applies this rate to training state of 12 bytes per parameter (fp32 weights plus two Adam moments). Models of about one billion parameters already need longer than the notice, and a 7B model needs more than ten minutes, so for these jobs the free notice cannot save a checkpoint and

Table 1: Time to write a checkpoint from one EKS node to S3 at the measured rate. Training state is 12 bytes per parameter.

Model	Parameters	Training state	Time to S3	Fits in 2 min?
CIFAR-10 CNN (this study)	0.3M	4 MB	under 1 s	yes
ResNet-50	25.6M	307 MB	3 s	yes
BERT-large	340.0M	4.1 GB	36 s	yes
1.3B-parameter LLM	1.3B	15.6 GB	136 s	no
7B-parameter LLM	7.0B	84.0 GB	735 s	no

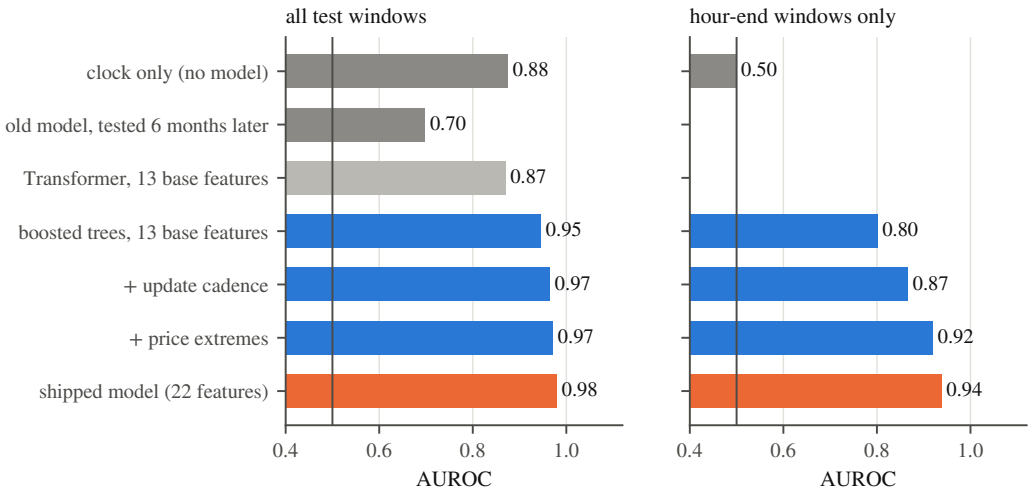


Figure 2: Test AUROC against controls. The clock-only rule is constant inside hour-end windows, so it scores 0.5 there. The old model was trained on January to April data and tested in October.

something must act earlier. The argument does not depend on the CIFAR-10 testbed. The simulator sees a job only through its node count and its checkpoint time, and Table 1 spans five orders of magnitude in model size, so the same analysis applies to any checkpointable job. The price model was trained on 30 instance types, including GPU and accelerator families, in eight regions.

3.2 Is the risk model a predictor?

The model scores whether a pool's price will rise by more than 1% within the next 15 minutes. This is a proxy, since real reclaim labels are not public, and Section 3.4 tests it against real reclaims. We trained on 768 price series (29 instance types in 8 regions, 90 days) with 20,819 positive training windows, and evaluated on the last days of every series, which nothing else saw. Scaling and calibration use training and calibration slices only.

Two controls matter. AWS now publishes Spot prices about once an hour, so every price spike lands on the hour, and a rule that only knows "the top of the hour is near" reaches AUROC 0.876 (lift 4.0) with no model at all. The model we first shipped, trained on January to April data, scores AUROC 0.699 (lift 2.2) on the same October windows, and 0.656 on the very instance types it was trained on. Its earlier headline lift did not survive six months. Two non-learned predictors fare no better. A volatility rule (recent price spread over its mean) scores AUROC 0.43, and the Spot Advisor interruption band [Amazon Web Services, 2026f] scores 0.65, which is a prior across instance types and carries no timing signal. Skill must therefore be judged against the clock and among the hour-end windows where a spike is possible. The retrained gradient-boosted model reaches lift 101.4 and AUROC 0.980 overall and 0.939 among hour-end windows (Figure 2). Across five seeds the lift ranges only from 98.5 to 101.4. It is calibrated (Brier score 0.00136 against 0.00147 for always predicting the base rate), and retraining at 5 successive cut-off dates keeps AUROC between 0.974 and 0.985, even with only 37.9 days of training data.

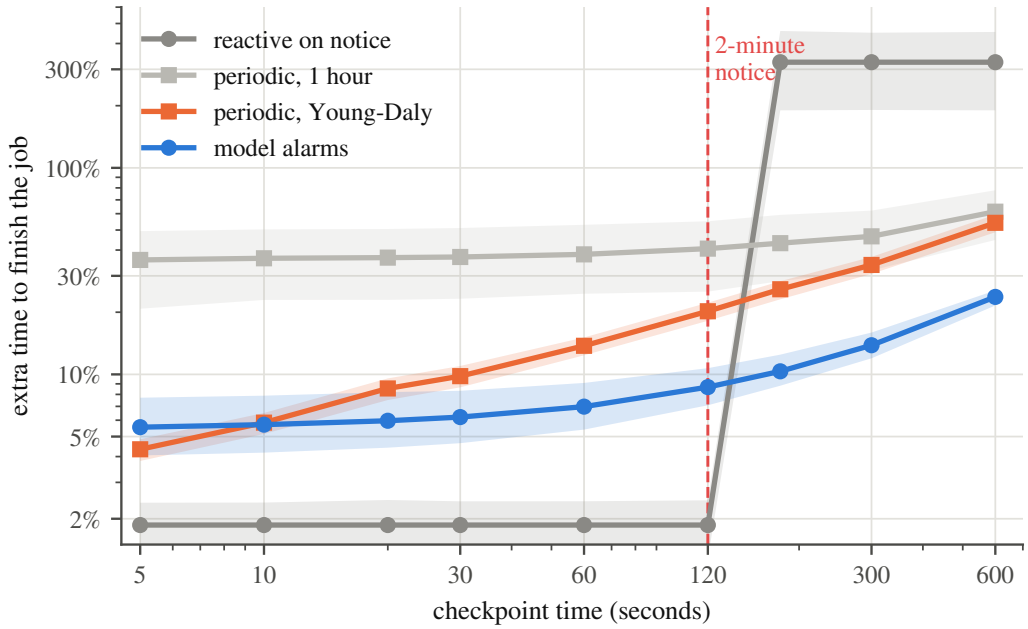


Figure 3: Extra time to finish a 15-node job against checkpoint time. Shading is a 95% interval resampled by whole days. Reacting to the notice is best until a checkpoint no longer fits in 120 seconds. After that, model alarms stay lowest.

A multi-seed ablation (Appendix A) settles two features that earlier single runs could not. Cross-AZ price divergence and the Spot Advisor interruption rate change lift by under one point, inside the seed spread, for both boosted trees and a Transformer. What helps is how often AWS updates a price and where the price sits relative to its recent range, which raise boosted-tree lift from 14.8 to 93.2. The Transformer follows the same pattern but is weaker and far less stable across seeds.

3.3 Does prediction pay?

We replay the last nine days of eu-north-1 prices for the five instance types and three AZs of our fleet. Interruptions are the price spikes in that trace (66 events). A synchronous job needs 12 hours of compute and is stopped by an event on any of its nodes. Checkpoints pause the job for C seconds, restarts cost two minutes, and every policy is run from many start times. A policy sees only what it could see at the time. The model policy alarms when the calibrated risk of any node crosses a threshold, checkpoints and migrates. The threshold is the 10% alarm budget chosen on the calibration slice by running the same simulation there, never on the test slice. Baselines are no protection, reacting to the notice, checkpointing every hour, checkpointing at the Young-Daly interval [Young, 1974, Daly, 2006] computed from the calibration slice's event rate, a variant that sets that interval from the Spot Advisor's published interruption bands instead (slowdown 1.60 at 300 seconds, worse than using observed events), and an oracle with five minutes of perfect warning. The threshold we deploy is 0.05, the 10% alarm budget for this fleet. The default of 0.65 in our first version never fired against this model's calibrated scores, which was a configuration error and not a design choice.

Figure 3 and Table 2 show the result. Reacting to the notice is near optimal while a checkpoint fits inside two minutes (2% extra time) and collapses to no protection beyond it. The model matches Young-Daly for fast checkpoints and pulls ahead as checkpoints grow. At 300 seconds model alarms add 14% against 34% for Young-Daly, a difference of 0.200 in slowdown (interval 0.178 to 0.222, paired and resampled by day) that favours the model on 100% of the 8 days. At 60 seconds the gain is 0.068 and at 10 seconds the two tie. The old model, at the same budget, is much worse than periodic checkpointing (slowdown 2.48 at 300 seconds), which shows that the model's quality is

Table 2: Slowdown against ideal (1.0 is no loss) for a 15-node job, by checkpoint time. Bold marks the best non-oracle entry where the model wins.

Policy (15 nodes)	10 s	60 s	300 s
No protection	4.25	4.25	4.25
Reactive on the 2-minute notice	1.02	1.02	4.25
Periodic, every hour	1.36	1.38	1.47
Periodic, Young-Daly interval	1.06	1.14	1.34
Alarms from the old model, same budget	1.82	1.91	2.48
Alarms from the retrained model	1.06	1.07	**1.14**
Oracle, 5 minutes of perfect warning	1.01	1.01	1.03

what makes prediction useful. At the budget used it catches 74% of the events with 9.5% precision, against 11% and 0.8% for the old model.

Recall against false alarms. The alarm budget sets the trade-off, so sweeping it varies recall and false-alarm rate for the real model (Appendix C). With too few alarms the model misses almost every event and the job is barely protected (slowdown 4.01 at a 0.2% budget). At a 10% budget it catches 74% of events at 10% precision and reaches 1.14, the best point. Doubling the budget raises recall to 92% but drops precision to 4%, and the extra checkpoints and restarts push slowdown back to 1.39. We also added an idealized predictor with chosen recall and precision on top of the Young-Daly schedule (slowdown 1.34 alone). At 95% recall it improves on Young-Daly only when precision is high (1.29 at 90%, 1.34 at 60%), and at 5% precision it is far worse (2.55). With 300 second checkpoints and 15 nodes every false alarm is expensive, so precision matters as much as recall, and a predictor that keeps the periodic schedule underneath pays for both.

Other regions. We repeated the replay in the seven other regions with the same protocol (Appendix D). Among the four regions with at least 20 events the model wins in two (eu-north-1 and ap-southeast-1), ties in two (us-east-1 and us-east-2), and never loses significantly at 300 seconds. The remaining regions had one to seven events in the test slice, which is too few to separate any policies.

3.4 Do price-based predictions anticipate real reclaims?

The proxy label and the simulator both treat a price spike as an interruption. To test that, we need real reclaims. AWS records a `BidEvictedEvent` in CloudTrail whenever it reclaims one of our instances [Amazon Web Services, 2026a]. Our account held two such events, a c5.xlarge and an m5.large in eu-north-1b, each reclaimed minutes after launch. Neither the retrained model nor the old one raised any warning. Neither pool had a price spike.

Two events cannot support a rate, so we produced more on purpose. A fleet of tiny Spot instances, one per instance type and AZ pool, was kept alive over several sessions in one day and each reclaimed instance was replaced, with every instance capped to self-terminate. Every five minutes we also recorded AWS's Spot placement score for each type and AZ [Amazon Web Services, 2026c]. Of 140 instances 90 were reclaimed, the median after 11 minutes of life, and another 131 launches failed outright for lack of capacity. Reclaims came from 4 of the five instance types, so pool identity explains much of the pattern. The unit of analysis is an instance and a five-minute step, labelled positive if AWS reclaims that instance within the next 15 minutes, which gives 124 positive steps among 1,446. Reclaims hit young instances, so we include instance age as a control, and we also report each predictor inside the first 30 minutes of an instance's life (512 steps, 93 positive). Table 3 shows how well each causal predictor ranks the steps.

The price-history model has no skill on real reclaims (AUROC 0.54, and 0.52 in the first 30 minutes), and the price-spike proxy does not coincide with them. Capacity signals do. AWS's placement score reaches 0.79 overall and 0.79 among young instances, so it is not merely tracking age, and the reclaim rate a pool showed in other hours reaches 0.90. We cannot tell the last two apart yet, since pools differ in their placement scores. A logistic model on placement, age and price risk reaches 0.81, a little above the placement score alone but well inside the intervals, so we cannot claim it is better. On pools it never saw during fitting, placement and age together reach 0.81 (interval 0.74

Table 3: AUROC for anticipating a real reclaim within 15 minutes, with a 95% interval resampled by whole hours, and the same restricted to the first 30 minutes of an instance's life. The last row is not causal and asks whether the price-spike proxy coincides with real reclaims.

Predictor	AUROC	95% interval	First 30 min
Price-history model (retrained)	0.54	[0.46, 0.62]	0.52
AWS Spot placement score	0.79	[0.74, 0.84]	0.79
Reclaim rate of the pool (leave one hour out)	0.90	[0.86, 0.93]	0.88
Instance age, younger is riskier (control)	0.77	[0.71, 0.84]	0.75
Logistic model on placement, age, price risk	0.81	[0.74, 0.86]	0.81
Price spike in the next 15 min (not causal)	0.50	[0.50, 0.50]	0.50

to 0.87). The intervals are still wide because the record covers one region and one day.alAurocReb AWS's own early warning, the Rebalance Recommendation, was recorded for the sensors launched after we began capturing it. It preceded ealRebWarned of ealRebCovered reclaims, a median of ealRebLead minutes ahead, and was also sent to ealRebFalse of the ealRebUnrec instances that were never reclaimed. Used as a predictor it reaches AUROC ealAurocReb (interval ealLoReb to ealHiReb).

i This does not overturn Section 3.3, which is correct about what it measures, but it limits it. The benefit shown there holds if reclaims follow price spikes, and in this region they mostly did not. A deployed system should combine or replace the price features with capacity signals such as the placement score.

3.5 Running the full path on a real cluster

We ran the operator, the retrained model served live, and training pods on a three-node EKS cluster in eu-north-1. Prices reach the service through one S3 object that a feeder refreshes, because the pod role can read only the checkpoint bucket. To trigger the predictive path on demand we set a job's threshold just below the live score. With the small CIFAR-10 model the operator saw a live score above the threshold, wrote a request, received the pod's acknowledgement after 2.1 seconds, deleted the pod, and the replacement was ready and resumed from the checkpoint after 12 seconds. With a 2 GB checkpoint the acknowledgement took 34.3 seconds, and the replacement resumed from the full checkpoint. Two jobs ran at the same time on the three-node cluster, and the second, with a high threshold, ran undisturbed on another node throughout. The reactive path was tested earlier with an injected interruption warning, not one issued by AWS. We placed an event in the exact schema of AWS's Spot interruption warning on the Node Termination Handler's queue, and the handler drained a real node. This gave the same flow, with the checkpoint reaching S3 within a second of SIGTERM and the replacement pod resuming in about 75 seconds.

The cluster run found five defects in our own design that unit tests had missed. The pod role cannot delete objects, so confirming a flush by waiting for the trigger to disappear could never succeed, which led to the acknowledgement handshake. After a migration the operator re-migrated on every poll because the pool's score stayed high, so the replacement pod never stayed up, which led to the cooldown. Polling for the trigger every 100 batches made the delay depend on batch time, so with a 2 GB model the operator timed out before the pod looked, which led to polling by wall clock. Uploading through a temporary key and copying it over the real one, meant to make the write atomic, added about 18 seconds on a 2 GB checkpoint and produced an object that failed an ETag check on download, and an S3 upload is already atomic, so we upload directly. Last, that failed download was read as "no checkpoint" and would have restarted training from scratch, so a download error now retries and then fails loudly.

4 Limitations

The strongest limit is the label. The model and the simulator use price spikes as a proxy, and Section 3.4 shows the proxy is a poor guide to real reclaims in our data. The real-label test covers one region, tiny instances, and one day, and sensors that are reclaimed within minutes of launch may behave differently from long-running training nodes. The benchmark covers nine days per region and the intervals, though resampled by day, rest on that short record. The simulator assumes

the notice always arrives, that a migrated node cannot be reclaimed for 15 minutes, and that a synchronous job is stopped by any node. The operator manages jobs individually and does not itself coordinate the nodes of one synchronous job. Checkpoint cost is a parameter, tied to model size through the measured S3 rate, not measured for each model. The live price sources that call the EC2 API need an IAM permission we could not grant, so the cluster run used an S3 feed.

5 Related Work and Conclusion

Varuna, Bamboo and Oobleck make training survive preemption through elastic rescaling, redundant computation or pipeline templates [Athlur et al., 2022, Thorpe et al., 2023, Jang et al., 2023]. CheckFreq and Gemini make checkpoints cheaper or faster [Mohan et al., 2021, Wang et al., 2023]. The optimal periodic interval is classical [Young, 1974, Daly, 2006], and we use it as the strongest non-learned baseline. Learned failure and interruption prediction for systems [Lin et al., 2018] is the closest line of work, and our contribution is to test such a predictor against real events. AWS also publishes signals of its own. The Spot placement score [Amazon Web Services, 2026c] estimates available capacity, and a Rebalance Recommendation [Amazon Web Services, 2026d] warns that an instance is at elevated risk of interruption, often before the two-minute notice, and is the natural free baseline for any learned predictor. Public archives such as SpotLake [Lee et al., 2022] collect Spot prices, placement scores and interruption-frequency bands across regions, but not the reclaims of individual instances, which is why we produced our own labels.

Reacting to the notice is enough while checkpoints fit inside it. Once they do not, which happens for models beyond about a billion parameters, a good predictor can beat the best periodic policy by a wide margin. Our predictor is good at price spikes and not at real reclaims, while AWS's placement score looks promising. The next step is training on real reclaim labels with capacity features, and the sensor fleet and label pipeline in the released code are a start.

References

Amazon Web Services. Logging EC2 spot instance events with CloudTrail. Amazon EC2 User Guide, 2026a.

Amazon Web Services. AWS node termination handler. `https://github.com/aws/aws-node-termination-handler`, 2026b. Accessed Aug. 2026.

Amazon Web Services. Spot placement score. Amazon EC2 User Guide, 2026c. `https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-placement-score.html`.

Amazon Web Services. EC2 instance rebalance recommendations. Amazon EC2 User Guide, 2026d. `https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/rebalance-recommendations.html`.

Amazon Web Services. Amazon EC2 Spot Instances. `https://aws.amazon.com/ec2/spot/`, 2026e. Accessed Aug. 2026.

Amazon Web Services. Amazon EC2 Spot Instance Advisor. `https://aws.amazon.com/ec2/spot/instance-advisor/`, 2026f. Accessed Aug. 2026.

Sanjith Athlur, Nitika Saran, Muthian Sivathanu, Ramachandran Ramjee, and Nipun Kwatra. Varuna: Scalable, low-cost training of massive deep learning models. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2022.

John T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, 2006.

Jason Dobies and Joshua Wood. *Kubernetes Operators: Automating the Container Orchestration Platform*. O'Reilly Media, 2020.

Insu Jang, Zhenning Yang, Zhen Zhang, Xin Jin, and Mosharaf Chowdhury. Oobleck: Resilient distributed training of large models using pipeline templates. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, SOSP '23, pages 364–380. ACM, 2023.

Sungjae Lee, Jaeil Hwang, and Kyungyong Lee. SpotLake: Diverse spot instance dataset archive service. In *IEEE International Symposium on Workload Characterization (IISWC)*, 2022.

Qingwei Lin, Ken Hsieh, Yingnong Dang, Hongyu Zhang, Kaixin Sui, Yong Xu, Jian-Guang Lou, Chenggang Li, Youjiang Wu, Randolph Yao, Murali Chintalapati, and Dongmei Zhang. Predicting node failure in Cloud service systems. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE '18, pages 480–490. ACM, 2018.

Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. CheckFreq: Frequent, fine-grained DNN checkpointing. In *Proceedings of the 19th USENIX Conference on File and Storage Technologies*, FAST '21, pages 203–216. USENIX Association, 2021.

John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. Bamboo: Making preemptible instances resilient for affordable training of large DNNs. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023.

Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, T. S. Eugene Ng, and Yida Wang. GEMINI: Fast failure recovery in distributed training with in-memory checkpoints. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, SOSP '23, pages 253–269. ACM, 2023.

John W. Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9):530–531, 1974.

A Feature ablation

Table 4: Test lift and AUROC by feature set, mean of three seeds. Hour-end AUROC is measured among the windows where a spike is possible.

	Boosted trees			Transformer	
Features	Lift	AUROC	Hour-end AUROC	Lift	AUROC
13 base features	14.8	0.946	0.802	9.8	0.870
+ cross-AZ price divergence	15.3	0.945	0.802	10.8	0.876
+ Spot Advisor interruption rate	14.4	0.944	0.806	9.9	0.869
+ update cadence	26.7	0.966	0.867	21.6	0.943
+ price extremes	48.5	0.972	0.920	35.5	0.949
all 19 features	93.2	0.978	0.936	50.3	0.960
all 28 features	99.3	0.981	0.941		

B Rolling-origin evaluation

Table 5: The model retrained from scratch at five cut-off dates and tested on each following nine-day block.

Test block	Train days	Positives	Lift	AUROC	Hour-end AUROC
10-04 to 10-13	74	2,947	98.6	0.978	0.938
09-25 to 10-04	65	6,883	68.1	0.982	0.948
09-16 to 09-25	56	1,621	127.8	0.977	0.936
09-07 to 09-16	47	2,549	104.8	0.985	0.948
08-29 to 09-07	38	3,025	67.6	0.974	0.925

Table 6: The model at each alarm budget on the October test slice: recall and precision against price-spike events, and slowdown for 15-node jobs with 300 second checkpoints. Intervals resample whole days.

Alarm budget	Recall	Precision	Alarms per node-day	Slowdown, 95% interval
0.05%	3%	45%	0.1	4.16 [3.11, 5.44]
0.1%	3%	50%	0.1	4.16 [2.85, 5.35]
0.2%	3%	40%	0.1	4.01 [2.87, 5.21]
0.5%	11%	36%	0.4	2.22 [1.78, 2.61]
1%	27%	36%	1.1	1.70 [1.43, 1.98]
2%	39%	25%	2.3	1.28 [1.18, 1.39]
5%	61%	18%	5.1	1.15 [1.12, 1.19]
10%	74%	10%	11.6	1.14 [1.12, 1.15]
20%	92%	4%	31.9	1.39 [1.33, 1.47]
Young-Daly, no predictor				1.34 [1.31, 1.37]

Table 7: 15-node jobs with 300 second checkpoints. Difference is the slowdown removed relative to Young-Daly, so positive favours the model. Intervals resample whole days.

Region	Events	Model	Young-Daly	Difference [95% interval]	Read
eu-north-1	66	1.14	1.34	+0.200 [+0.178, +0.222]	model better
us-east-2	26	1.16	1.17	+0.015 [-0.057, +0.067]	tie
ap-southeast-1	22	1.10	1.16	+0.066 [+0.015, +0.134]	model better
us-east-1	22	1.15	1.13	-0.015 [-0.075, +0.051]	tie
us-west-2	7	1.06	1.07	+0.009 [-0.014, +0.035]	too few events
eu-central-1	3	1.06	1.06	+0.000 [-0.061, +0.064]	too few events
eu-west-1	1	1.02	1.03	+0.005 [-0.006, +0.017]	too few events
ap-south-1	1	1.01	1.15	+0.137 [+0.130, +0.144]	too few events

C Operating points of the model

D Other regions

E Reproducibility

All numbers in this paper are generated from the result files by `Paper/v2/build_numbers.py` and `build_tables.py`. Code, data pipeline, simulator, cluster manifests and the sensor fleet are in the project repository.

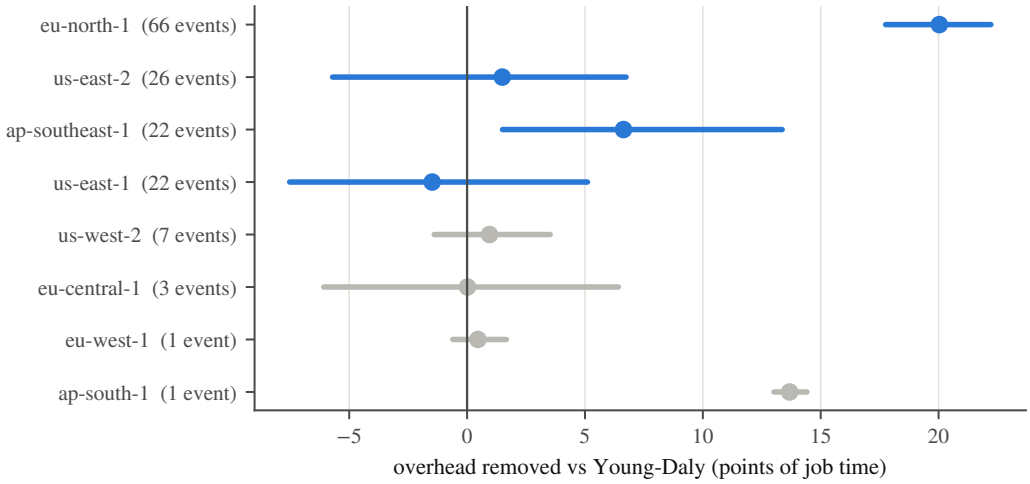


Figure 4: Overhead removed relative to Young-Daly by region. Grey marks regions with fewer than 20 events.